

VULNERABILITY OPERATIONS CENTER

The VOC Platform

An autonomous, agentic attack-path intelligence system for vulnerability management — full technical documentation & design rationale.

FastAPI + React SPA

Deterministic-first engine

6 AI agent roles

Tool-using ReAct detection

Graph-grounded, 0 hallucination

litellm multi-provider

voc

Contents

0 · Executive summary — what this platform is, in one page	3
1 · The idea & the problem — why it exists, the core thesis	4
2 · Technology stack — every dependency and why it's there	5
3 · System architecture — deterministic-first / AI-additive	6
4 · The deterministic core — GRS, CMDB, capabilities, graph, oracle	7
5 · The AI agent layer — the 6 agents, in detail	11
6 · The tool-using detection agent — ReAct loop & path scoring	13
7 · Evaluation & quality guarantees — the eval harness	15
8 · API surface — REST & SSE endpoints	16
9 · Frontend — the React SPA	17
10 · Data model & datasets	18
11 · Running & operating the platform	19
12 · Where we can improve — tech-stack & roadmap	20
13 · Appendix — glossary & configuration reference	22

0 · Executive summary

The VOC (Vulnerability Operations Center) turns a flat list of scanner findings into a **prioritized, business-aware, attacker's-eye view** of an enterprise: not "here are 33 CVEs sorted by CVSS", but "here is the exact chain an attacker walks from the internet to your SWIFT gateway, why it works, who owns each hop, and what to fix first."

2

SERVICE TIERS
(FASTAPI +
REACT)

6

AI AGENT
ROLES

4

ENTERPRISE
DATASETS

0

HALLUCINATED
HOPS
(ENFORCED)

73

BACKEND
TESTS
GREEN

The platform is built on one central design conviction, which every part of it obeys:

The governing principle: deterministic-first, AI-additive

A rule-based, math-only engine computes risk scores and enumerates attack paths **with no LLM and no network access**. That output is the trust anchor. The AI agents sit *on top* — they explain, narrate, cross-reference and prioritize, but they can never invent a host, a CVE, or a network hop that the deterministic graph doesn't already support. Pull the AI layer out entirely and the platform still produces byte-identical risk scores and attack paths.

This is what makes an *agentic* security tool trustworthy: the creative, fallible part (the LLM) is fenced inside a verifiable, reproducible part (the graph + rules). The rest of this document explains how every layer implements that fence.

Layer	What it does	Needs an API key?
Deterministic core	GRS risk scoring, CMDB parsing, capability classification, reachability graph, attack-path discovery, held-out oracle grading	No — pure Python/math
AI agent layer	Capability enrichment, autonomous path detection, narration, correlation, compliance, executive synthesis, chat	Yes — degrades cleanly without
API	FastAPI JSON + Server-Sent-Events streaming	No
Frontend	React SPA: dashboards, interactive attack graph, findings table, AI analyst chat	No

1 · The idea & the problem it solves

1.1 The problem with CVSS-sorted vulnerability lists

A vulnerability scanner produces hundreds or thousands of findings, each with a CVSS severity score. Teams triage them top-down by CVSS. This fails in a specific, damaging way: **CVSS measures how bad a vulnerability is in the abstract, not how bad it is *here, in this network, on this asset*.**

The motivating example (baked into the platform's test data)

A CVSS **10.0** RCE on an isolated, admin-only internal VM (VCENTER01) is, in practice, nearly unreachable — an attacker has almost no way to contact it. A CVSS **7.0** flaw on an internet-facing web portal that chains toward a regulated database is far more dangerous. Naive CVSS sorting ranks them backwards. The VOC's scoring engine scores the "9.8" at **26.9 (TRACK)** and the "9.1" at **65.7 (ACT)** — a deliberate, defensible reordering.

1.2 The core thesis

Risk is not a property of a vulnerability. It's a property of a vulnerability *on a specific asset, reachable via a specific network path, that chains to something the business cares about*. To score risk correctly you must model:

- **Exploitation likelihood** — is anyone actually exploiting this in the wild? (EPSS + CISA KEV)
- **Asset criticality** — how much does this asset matter to the business and to regulators? (ACW + DORA scope)
- **Reachability** — can an attacker actually get to it, and from where? (exposure tier, network graph)
- **Toxic combinations** — does this finding, combined with others, open a path to a "crown jewel"? (attack-graph blast radius)

1.3 From scoring to autonomous discovery

Scoring each finding in context is step one. The platform's real intellectual core is step two: **automatically discovering the attack paths** — the multi-hop chains from an internet entry point to a business-critical asset — the way a red-team operator would reason about them, and doing so *from the environment model alone*, never from a pre-written answer key.

The "no answer key" invariant

Early versions let the LLM read the documented attack paths and hand them back — impressive-looking, but a parlor trick, not detection. The platform was redesigned so the documented paths are **held out** as a hidden oracle used *only to grade* the engine. No classifier, graph, or agent ever sees them. When the engine rediscovers a documented path, that is a genuine detection, and it is measured as such.

1.4 History (how we got here)

The platform began as a Streamlit prototype (now in `_legacy/`). It was rebuilt into the current FastAPI + React architecture, then hardened across a series of documented initiatives: severing the answer key, converting all datasets to a relational CMDB, and a six-phase "attack-path intelligence" brief (Phases 0–5)

that added an evaluation harness, a hybrid classifier, an environment-agnostic graph, the tool-using detection agent, calibrated path scoring, and production hardening.

2 · Technology stack

Deliberately small and mainstream. Every dependency earns its place; nothing exotic.

2.1 Backend (Python)

Component	Technology	Role & why chosen
Web framework	FastAPI (pinned 0.115.6)	Async, typed, automatic OpenAPI docs. Serves both JSON REST and SSE streams. Pinned because 0.139.0 was a bad build.
ASGI server	uvicorn	Runs the app; <code>--reload</code> in dev.
Data / scoring	pandas	The findings CSV is loaded, GRS-enriched, and sorted as a DataFrame — the canonical in-memory findings table.
Graph engine	networkx	The reachability DiGraph: hosts + INTERNET node, edges = network/credential/domain reachability. Path enumeration and shortest-paths run on it.
Validation	pydantic v2	Request bodies and every structured LLM output (capability extraction) are Pydantic-validated; malformed LLM JSON is dropped, not trusted.
LLM access	litellm (≥1.51)	The <i>only</i> LLM dependency. One uniform <code>completion()</code> call fronts 7 providers — no vendor SDK anywhere.
Persistence	SQLAlchemy 2.0 + SQLite	Analysis runs are persisted. SQLite by default (zero-ops, file-based); swap to Postgres via one <code>DATABASE_URL</code> env change.
Config	python-dotenv	Reads API keys + tunables from <code>.env</code> .

2.2 Frontend (TypeScript)

Component	Technology	Role
Framework / build	React 18 + Vite 5 + TypeScript 5	SPA; Vite dev server on :5173 proxies <code>/api</code> to the backend.
Styling	Tailwind CSS 3	Utility-first; theme-aware (light/dark) with shared shadow/color vars.
Server state	@tanstack/react-query	All API reads (caching, refetch, loading states).
Tables	@tanstack/react-table	The findings table (sort/filter, click-to-drill drawer).
Attack graph	Cytoscape.js + fcose + navigator	The interactive attack-path graph (canvas rendering + DOM overlay cards). Replaced an earlier React-Flow implementation.
Charts	Recharts	KPI charts on the dashboards.
Routing / motion / icons	react-router-dom, framer-motion, lucide-react	Navigation, animations (respecting reduced-motion), iconography.

2.3 LLM providers (all via litellm)

Provider	Env var	Default model	In fallback chain
Mistral primary today	MISTRAL_API_KEY	mistral-large-latest / small-2506 / medium-2505	1st
Groq	GROQ_API_KEY	llama-3.3-70b-versatile	2nd
Google Gemini	GEMINI_API_KEY	gemini-2.0-flash	3rd
Anthropic	ANTHROPIC_API_KEY	claude-opus-4-8	4th
OpenAI	OPENAI_API_KEY	gpt-4.1	5th
DeepSeek / xAI	DEEPSEEK_API_KEY / XAI_API_KEY	deepseek-chat / grok-4	6th / 7th

Why a provider abstraction, not one vendor?

The platform must never hard-depend on a single LLM vendor. Every agent role has a *preferred* provider plus an ordered fallback chain, tried automatically if the preferred key is missing or fails. The current deployment runs **Mistral only** (free tier), with per-role models matched to each model's free-tier throughput — but switching or mixing vendors is a config change, not a code change.

2.4 Infrastructure & tooling

Docker + docker-compose (two services + nginx for the built frontend), a `dev.sh` runner for local two-service startup, and `pytest` for the test suite (73 passing / 1 skipped). No external scanning or threat-intel APIs are called anywhere — no Shodan, VirusTotal, NVD, Censys, or Nmap. All vulnerability and asset data is static and local.

3 · System architecture

3.1 The two-half pipeline

Everything flows through one orchestrator (`agents/orchestrator.py`) split into two explicit halves. The split is the architecture:

① **compute_deterministic()** — always runs · token-free · no DB · no API key

Ingest findings CSV + compute GRS → parse CMDB → regex capability classification → build reachability graph + discover paths → load held-out oracle (grading only)

▼ if a provider key is configured

② **run_ai_layer()** — spends tokens · mutates result in place · cancellable at every boundary

Capability Extractor (hybrid re-classify → rebuild graph) → Analyst Detection Agent → Discovery → Correlation → Compliance → Triage synthesis

Graceful degradation is a guarantee, not a hope

With no provider connected, half ② is skipped and half ① produces exactly what it always produces — byte-identical risk scores and attack paths. Each AI agent is individually wrapped in try/except: a provider failure (rate limit, timeout, bad JSON) preserves the deterministic layer and logs a warning rather than failing the run. This is verified end-to-end by an automated test.

3.2 Request & persistence flow

- A plain `GET` (e.g. `/api/overview`) returns already-computed state — it never spends a token.
- Only `POST /api/analyze` and `POST /api/chat` trigger LLM spend. Both stream progress over **Server-Sent Events (SSE)** and are guarded by an inbound rate limiter *before* the stream opens.
- Analysis runs are persisted to the database (`analysis_run` table) keyed by dataset, so run history, trends, and duplicate-detection never mix one enterprise's data into another's.
- The AI agents' output — including the detection agent's paths and the per-run token/cost trace — is folded into the persisted `discovery` blob and split back out on rehydrate, so no schema migration was needed to add it.

3.3 Directory map

```
backend/  
  core/          GRS engine, CMDB, capability classifier, attack graph, oracle, providers,  
config            
  agents/       orchestrator + 6 agent roles + the ReAct detection loop & tools  
  api/          FastAPI routes, SSE streams, serializers, in-memory state, rate limiter  
  db/           SQLAlchemy models, session, repository  
  data/         per-enterprise CMDB CSVs + vulnerabilities.csv + held-out oracle + cache  
  eval/         evaluation harness + regression/leakage/hardening tests  
  tests/        rediscovery acceptance test  
frontend/  
  src/features/ one folder per page (overview, findings, attackpaths, correlation,  
compliance, teams, trends, analyst, settings, logs)
```

src/lib/	api client, types, hooks, report builders, theme, toast
_legacy/	the original Streamlit prototype (kept for reference)

4 · The deterministic core

Five subsystems, zero LLMs, zero network. This is the trust anchor the whole platform is built to protect. LLM agents may *explain* these outputs; they never compute or override them.

4.1 The Ghrab Risk Score (GRS) engine

`core/risk_engine.py` implements a context-aware risk formula that replaces naive CVSS sorting. Every finding gets a GRS from 0–100, an action band, and an SLA.

```
GRS = min(100, ImpactScore × ExposureTier × CCF)

ImpactScore = 10 × ( 0.30·CVSS severity + 0.15·EPSS likelihood (as scanned) (EPSS×10) + 0.15·KEV confirmed exploit (KEV: 10/0) + 0.20·ACW asset-criticality (0–10) + 0.20·TCM toxic-combo / blast radius (0–10) )

ExposureTier ∈ { 1.30 internet-facing · 1.15 adjacent · 0.90 internal · 0.50 restricted · 0.15 isolated } ← a GATING multiplier

CCF = Compensating Controls Factor, 0.5–1.0 (only verified controls reduce it)
```

Why exposure is a multiplier

Reachability gates everything. An attacker who can't reach an asset has near-zero contact frequency regardless of severity. An isolated CVSS 10 (×0.15) and an internet-facing CVSS 7 (×1.30) differ by an **8.7× swing from exposure alone** — exactly the reordering the platform exists to produce.

The five factors, sourced

CVSS from the scan; EPSS probability + CISA KEV membership per-CVE (hand-enriched tables, keyed by CVE — "pull live from FIRST.org in production" is a documented caveat); ACW from asset criticality tiers; TCM derived from the finding's position in the documented attack chains.

Action bands (SSVC-aligned) & the DORA overlay

GRS	Band	Meaning	Default SLA
80–100	IMMEDIATE	Likely exploitation path to a crown jewel, low barrier	24–72h emergency change
60–79	ACT	Exploitable + reachable, meaningful business impact	7 days
40–59	ATTEND	Real risk, exposure/blast partially contained	30 days
20–39	TRACK*	Elevated interest, low current exploitation signal	90 days / next cycle
0–19	TRACK	Low priority under current conditions	Monitor only

DORA overlay: any finding on an asset in scope for a "critical or important function" (CIF) has its SLA capped at 30 days regardless of computed band, reflecting EU DORA RTS Article 10. This regulatory layer is applied deterministically, not left to an LLM.

4.2 The CMDB — a relational asset/network model

The Configuration Management Database (`core/cmdb.py` , `cmdb_store.py` , `cmdb_schema.py`) is the environment model every downstream stage grounds against. All four datasets are now **CSV-backed relational CMDBs** in ServiceNow-style CI tables:

- **CIs (Configuration Items)** — servers, network gear, databases, cloud resources — each with an ID, name, zone, platform, owner team, and criticality.

- **net_reachability** — the *authoritative* zone-to-zone reachability table. Each rule has a status: `Intended` / `Excessive` (traversable) or `Should-Not-Exist` (a hard veto — no edge of any kind may connect those zones).
- **Relationships, dependencies, credential-reuse** — which CIs depend on which, and where credentials are valid (the raw material for lateral-movement edges).

Referential integrity is validated, loudly

`GET /api/cmdb/validate` (and a boot-time check) reports dangling references, orphan CIs, zones missing from the reachability base, duplicate CI IDs, unknown relationship types, and findings whose CI doesn't resolve. The acmebank dataset intentionally seeds defects to prove the validator fails loud rather than silently. This directly fixed a class of bug where a malformed CMDB silently loaded as empty and starved every downstream stage of grounding.

4.3 The capability classifier — findings → attacker capabilities

`core/capability.py` reads each finding and classifies it into ATT&CK-aligned **effects** — what an attacker *gains* by exploiting it — plus preconditions. This is what lets the graph reason about movement rather than just "a CVE exists here."

Effect	Meaning
<code>initial_access</code>	A first foothold from outside a boundary (an entry point)
<code>rce_system</code> / <code>rce_user</code>	Remote code execution as SYSTEM/root, or at lower privilege
<code>credential_theft</code>	Yields credentials usable elsewhere (enables lateral movement)
<code>privilege_escalation</code> / <code>cloud_priv_esc</code>	Elevates on-host, to domain admin, or to cloud admin
<code>segmentation_break</code>	Opens a network boundary between zones (enables a cross-zone hop)
<code>impact</code>	Terminal business impact (data exfiltration, settlement fraud, ...)

The classifier is a **hybrid** (see §5.2): a fast regex pass, optionally merged with an LLM pass. Crucially, the LLM never names hosts or zones — those pivots stay derived deterministically — so the LLM cannot introduce an out-of-scope host. Hallucination is **0 by construction**, not by luck.

4.4 The attack graph & path discovery

`core/attack_graph.py` builds a directed reachability graph and enumerates attack paths on it.

- **Nodes:** a synthetic `INTERNET` node plus every host CI.
- **Edges:** network reachability (from the authoritative `net_reachability` rules, each carrying its rule ID + enabling QID for auditability), plus credential-reuse and domain-admin edges derived from capabilities. A `Should-Not-Exist` rule **veto**es any edge between those zones.
- **Discovery:** enumerate `INTERNET → crown-jewel` walks, rank them by a weighted blend of target business value, peak finding risk, lateral blast radius, and a mild long-chain penalty.

Environment-agnostic by design (Phase 2)

Crown jewels, entry points and tier adjacency are derived from the CMDB's **trust levels, criticality labels, platform strings and dependency edges** — *not* from hardcoded VLAN numbers. Earlier versions hardcoded "VLANs 21–30 are the domain" constants; those are gone. The engine now generalizes to any environment or numbering scheme, with every threshold/token in `core/config.py`.

4.5 The held-out oracle

`core/oracle.py` + `core/graph.py` load the documented "ground-truth" attack paths from `data/oracle/`. This is the answer key — and it is **quarantined**. Nothing in the detection pipeline imports it; it exists *only* to grade how well the engine and agents rediscover the real paths. A dedicated test (`test_no_oracle_leakage.py`) enforces that quarantine in CI.

5 · The AI agent layer

Six roles, one shared base, one provider abstraction. Each agent explains or enriches deterministic ground truth — none is trusted to invent facts.

5.1 The provider abstraction & resilience

Every agent calls the LLM through one layer (`core/providers.py`) built on litellm — no agent touches a vendor SDK. That layer provides three things that make free-tier, multi-provider operation survivable:

Mechanism	What it does
Fallback chain	Each role has a preferred provider; if its key is missing or a call fails, the call falls through <code>mistral → groq → gemini → anthropic → openai → deepseek → xai</code> .
Client-side throttle	Same-provider (and same- <i>model</i>) calls are spaced by a minimum interval tuned to each free-tier's RPS cap — e.g. <code>mistral-small</code> allows ~5 RPS (0.25s spacing) but <code>mistral-medium</code> ~0.42 RPS (2.75s). Prevents tripping RPM caps mid-run.
Retry + backoff + timeout	A <code>429</code> or a timeout is retried on the <i>same</i> provider with exponential backoff (honoring <code>Retry-After</code>) before falling through. A hard per-request wall-clock timeout stops a hung provider from stalling the whole run.

All of it is tunable via env (`LLM_MIN_INTERVAL_SEC` , `LLM_MAX_RETRIES` , `LLM_REQUEST_TIMEOUT_SEC` , ...) with no code change. Every call is recorded in `core/call_log.py` — model, provider, tokens, cost, latency — producing a **real** per-run accounting trace (not an estimate) attached to each analysis.

5.2 The six agent roles

Agent	What it does	Output	In bulk run?
Capability Extractor capability	Re-reads each finding's full text with the LLM to catch capabilities the regex misses. Merged with the regex pass; never names hosts. Pydantic-validated, vocab-filtered, content-hash cached at temp 0.	Structured effects / grants per finding	Yes (1st)
Analyst Detection Agent attack_path	The headline agent. Given only grounding + capabilities (no candidate list, no oracle), it <i>autonomously reasons</i> attack paths using graph tools, verifying every hop. See §6.	Grounded/plausible detected paths	Yes (2nd)
Discovery Agent attack_path + correlation	Takes the deterministically-enumerated chains and narrates each (headline, story, business impact, choke point, confidence, novelty); separately finds cross-path "toxic combinations."	Narratives + toxic combos	Yes (3rd)
Correlation Agent correlation	Cross-references the full findings set against the CMDB to surface non-obvious risk, top-risk teams, and mis-prioritized findings.	Correlations, team risk	Yes (4th)
Compliance Agent compliance	Reasons over compliance scope (PCI DSS, SWIFT CSP, EU DORA, CIS) + CDE/CIF facts to produce a posture briefing for an auditor/board.	Compliance briefing	Yes (5th)
Triage / Analyst Agent triage	Powers the executive one-paragraph synthesis (fed the other agents' conclusions so it can't contradict them) and the grounded AI-analyst chat.	Exec summary + chat	Synthesis: yes (6th)
Remediation Agent remediation	On-demand per finding (not bulk): enriches scanner remediation text into step-by-step actions, validation steps, risk-of-fix, and an effort estimate.	Remediation plan	No (on click)

Every agent output is grounding-verified

A hallucinated `path_id`, choke-point QID, involved QID, or compliance finding-ref would otherwise flow straight to the UI. After parsing, each agent **filters or flags** any reference that doesn't match a real path or QID from the deterministic ground truth. Bad JSON is brace-repair-tolerant and, failing that, dropped — the deterministic layer always stands.

5.3 Determinism & caching

Every role that produces **structured** output (capability, correlation, attack_path, remediation, compliance) samples at **temperature 0**, so identical inputs give identical outputs and content-hash caching is meaningful. Only `triage` — free-text prose nothing downstream parses — keeps a small documented non-zero temperature for more natural writing. Capability classifications are cached on disk keyed by (*finding content + model id*), so re-runs are token-free and reproducible.

6 · The tool-using detection agent

The platform's most sophisticated component: an autonomous agent that investigates and *verifies* attack paths hop-by-hop, instead of guessing them in one shot.

6.1 Why a ReAct loop, not a single prompt

A single-shot "here's the whole network, name the attack paths" prompt invites hallucination — the model fills gaps with plausible-sounding but fake hops. Instead the Analyst Detection Agent runs a bounded **ReAct loop** (`agents/agent_loop.py`): it reasons, calls a read-only tool against the *real* graph + CMDB, observes the true result, and iterates — building a path one verified hop at a time.

The tool contract

Tool	Returns (from the real graph/CMDB)
<code>list_entry_points()</code>	Internet/untrusted-reachable hosts
<code>list_crown_jewels()</code>	Crown-jewel targets with business value
<code>neighbors(host)</code>	Outbound edges: destination, kind, enabling QID
<code>finding_detail(qid)</code>	Full finding text + its extracted capability
<code>test_hop(from, to)</code>	Whether a real enabling edge exists, and which QID/relationship justifies it
<code>shortest_paths(from, to, k)</code>	Graph-computed candidate routes
<code>reachability_rule / relationships_of / credentials_valid_on / business_service_of</code>	Typed structured-CMDB queries (Phase 4) — read records on demand instead of depending on a truncated text context

Bounded absolutely — cost can't run away

The loop is provider-agnostic (a JSON-action protocol over plain completions — no vendor tool-calling dependency), seeded with precomputed shortest-path candidates so it usually finalizes in 2–3 turns, and capped three ways: a hard iteration cap (default 8 round-trips), an estimated-token budget, and a per-call token cap. Tools execute **locally** (token-free). Temperature 0. Cancellation is cooperative. Set `DETECTION_AGENT_ENABLED=0` to fall back to single-shot; no provider ⇒ neither path spends a token.

6.2 Verification & the three-label calibration (Phase 4)

Every reconstructed path is labeled — the agent is never allowed to silently assert an unconfirmable hop:

Label	Meaning
grounded	Every hop is a real enabling graph edge forming a connected INTERNET → target walk. Soundness 1.0 by construction.
plausible	All hosts + cited QIDs resolve in scope, but ≥1 hop the graph can't confirm. Kept, flagged <code>verified:false</code> , never presented as grounded.
rejected	No anchorable real host → not emitted at all. This is how hallucination stays 0.

The calibrated path score

Each path gets a score that is a weighted sum of four **inspectable** components (each exposed alongside the number, so an analyst can see *why* a path ranks where it does):

```
score = ( 0.35·reachability + 0.25·exploitability + 0.30·business_value + 0.10·chain_length
) × damping

reachability    = verified-hop ratio (real edges / total hops)
exploitability  = strongest signal across cited findings: KEV > EPSS > CVSS, else GRS/ACW
                  (in-dataset signals only – no external feeds)
business_value  = target crown-jewel value (zone criticality + DORA CIF + exposure)
chain_length    = shorter chains score higher (ideal_hops / max(hops, ideal))
damping         = <1 for plausible paths, so grounded always outranks equal plausible
```

All weights, damping and normalizers live in `core/config.py` — nothing is a magic inline number. Emitted paths are score-ranked, then capped (default 8).

7 · Evaluation & quality guarantees

The platform builds the ruler before changing the logic. Detection quality is measured, not asserted.

7.1 The eval harness

`backend/eval/detection.py` runs the full pipeline per dataset against the held-out oracle and reports, at both the whole-path (target) and individual-hop (edge) level:

P/R/F1
PRECISION · RECALL · F1
TARGET & EDGE LEVEL

1.0
SOUNDNESS
(MUST BE 1.0)

0
HALLUCINATION
(MUST BE 0)

AP/MRR
RANKING QUALITY

7.2 The hard invariants (enforced in CI)

Invariant	Guaranteed by
Oracle never leaks to any classifier/agent	<code>test_no_oracle_leakage.py</code>
Zero fabricated hops (hallucination = 0)	Real-host/QID/edge anchoring in <code>_build_path</code> ; eval asserts it
Grounded-soundness = 1.0	Grounded label only assigned to fully-verified walks
No-provider run is byte-identical to before	<code>test_phase5_hardening.py</code> end-to-end degradation test
No metric regression beyond tolerance	<code>test_detection_regression.py</code> (runs AI-off, so it never flakes on a rate limit)

7.3 Committed metrics (deterministic baseline vs AI-on)

Metric	ghrab base	ghrab AI-on	velon base	velon AI-on
target recall	0.667	0.667	0.833	0.833
target F1	0.571	0.571	0.588	0.588
edge recall	0.503	0.511	0.536	0.544
edge F1	0.548	0.553	0.574	0.582
soundness	1.000	1.000	1.000	1.000
hallucination	0.000	0.000	0.000	0.000
ranking MRR	1.000	1.000	1.000	1.000

An honest note on the numbers

Target recall isn't 100% and edge-level Jaccard wobbles slightly between the deterministic baseline and AI-on runs — because the agent sometimes picks a *different valid route* to the same crown jewel than the one the oracle documented, and Mistral isn't perfectly deterministic even at temperature 0. The team

deliberately did **not** game the metric to fake a "beat." The verifiable, reproducible win is **forbidden-hop elimination**: cross-zone edges that violate the `Should-Not-Exist` rules dropped from 10+ fabricated edges to **0**. That is what the CI gate asserts, alongside soundness=1.0 and hallucination=0. The committed baseline is always the deterministic (regex, no-key) run — never re-committed from an AI-on run.

8 · API surface

FastAPI, prefix `/api`. Reads are plain JSON GETs; the two token-spending endpoints stream over SSE.

8.1 Read endpoints (never spend tokens)

Endpoint	Returns
<code>GET /api/health</code>	Liveness probe
<code>GET /api/overview</code> · <code>/kpis</code>	Dashboard summary + KPI cards/charts
<code>GET /api/findings</code> · <code>/findings/{qid}</code>	Findings table; single-finding drill-in with capability
<code>GET /api/attack-paths</code>	Deterministic paths, AI-detected paths, toxic combos, documented overlay
<code>GET /api/graph</code>	The reachability graph payload for Cytoscape
<code>GET /api/verification</code>	Engine vs AI-agent rediscovery grade against the held-out oracle
<code>GET /api/correlation</code> · <code>/compliance</code> · <code>/teams</code>	Agent outputs + team risk rollups
<code>GET /api/cmdb/validate</code>	CMDB referential-integrity report
<code>GET /api/datasets</code> · <code>POST /api/datasets/select</code>	List enterprises / switch the active one
<code>GET/POST /api/settings/providers</code> · <code>/settings/agent</code>	Provider keys + per-role provider assignment
<code>GET /api/analyze/runs</code> · <code>DELETE .../{id}</code>	Persisted run history

8.2 Streaming / mutating endpoints (rate-limited, spend tokens)

Endpoint	Behavior
<code>POST /api/analyze</code>	Runs the full pipeline, streaming per-stage progress over SSE. Guarded by <code>RATE_LIMIT_ANALYZE_PER_HOUR</code> (default 6).
<code>POST /api/analyze/cancel</code>	Cooperative kill switch — stops at the next agent boundary.
<code>POST /api/analyze/reuse/{run_id}</code>	Rehydrate a persisted run without recomputing.
<code>POST /api/chat</code>	Grounded AI-analyst chat, streamed. Guarded by <code>RATE_LIMIT_CHAT_PER_MINUTE</code> (default 10).
<code>POST /api/findings/{qid}/remediation</code>	On-demand remediation enrichment for one finding.

Security posture — read this

The platform has **no authentication layer**: anyone who can reach the API can call it. The inbound rate limiter (`core/ratelimit.py`) is IP-keyed and in-memory — a stopgap against a runaway client, *not* a real quota system (IPs are spoofable / shared behind NAT). Real multi-tenant auth is a prerequisite for any shared or production deployment. See §12.

9 · Frontend — the React SPA

A single-page app in `frontend/`, one feature folder per page. All views render against the live backend; server state flows through React Query.

Page	What it shows
Command Center (Overview)	KPI cards, GRS/band charts, and the AI executive synthesis paragraph.
Findings	A TanStack table (sort/filter) with a click-to-drill slide-over drawer showing the full finding, its GRS decomposition, capability, and on-demand remediation.
Attack Paths	The interactive Cytoscape graph (INTERNET → crown jewel), ranked deterministic paths, AI-detected paths with grounded/plausible labels, replay, and toxic combinations.
Verification banner	Engine X/N and agent Y/N rediscovery vs the held-out truth.
Correlation · Compliance · Teams · Trends	Agent outputs, team risk rollups, and run-over-run trend history.
AI Analyst	Streaming grounded Q&A chat (SSE) over the CMDB + findings + discovered paths.
Settings · Logs	Provider keys, per-role model assignment, dataset picker; live agent/run logs.

Cross-cutting UX systems

A toast system, a ⌘K command palette, theme-aware shadow/color CSS vars (light/dark), global reduced-motion support, a shared overlay/drawer pattern, and animated count-up KPIs. Report builders (`lib/reportBuilders.ts`) assemble exportable kill-chain reports.

Rendering notes (hard-won)

The attack graph runs on Cytoscape.js (canvas + a DOM overlay for readable cards) after an earlier React-Flow build flashed white on hover due to a backdrop-filter bug. Recharts marks need `isAnimationActive={false}` to render under headless capture. The Cytoscape canvas color parser needs comma-form `rgb()`, not the space syntax.

10 · Data model & datasets

10.1 The four enterprise datasets

The platform ships with four fictional but internally-consistent enterprises. Which one is active is runtime-switchable; each is scoped separately in the database.

Dataset	Sector	Findings	Frameworks in scope
Ghrab Financial Group	Finance / banking	~32	PCI DSS, SWIFT CSP, EU DORA, CIS
Velon Health Systems	Healthcare	~32	HIPAA-style + CIS
Elihowa	(no virtualization tier)	~20	—
AcmeBank	Banking (validation reference)	~27	Intentionally seeds CMDB defects to prove the validator fails loud

10.2 Per-dataset file layout

data/<key>/	
vulnerabilities.csv	the scanner findings (QID, CVE, CVSS, category, description, CI_ID, ...)
cmdb/	
cmdb_ci_*.csv	Configuration Items (servers, netgear, databases, cloud)
cmdb_rel_ci.csv	CI-to-CI relationships
net_reachability.csv	authoritative zone-to-zone reachability rules
(Intended/Excessive/Should-Not-Exist)	
...	dependencies, credential-reuse
data/oracle/<key>_attack_paths.csv	HELD-OUT ground truth (QID → PATH-x-StepN). Grading only.
data/ghrab_risk_methodology.md	shared GRS methodology (org-independent)

CSV is the source of truth

All four datasets were converted from an earlier ServiceNow-style markdown format into these relational CSVs (the acmebank layout). The CSV directory now wins over any legacy flat markdown. Markdown files are retained on disk as a fallback/rendering source, but no active dataset selects the legacy parser. EPSS/KEV enrichment tables live in `core/risk_engine.py`, keyed by CVE, unioned across datasets (namespaces don't collide).

10.3 Persistence

SQLAlchemy 2.0 over SQLite (`data/voc.db`). One core table, `analysis_run`, stores each run's serialized output keyed by dataset. Swapping to Postgres is a single `DATABASE_URL` env change because all DB access goes through `db/repository.py`.

11 · Running & operating the platform

11.1 Local development

```
# Backend (from backend/) – port 8000
cd backend && uvicorn main:app --reload

# Frontend (from frontend/) – Vite dev server on 5173, proxies /api → 8000
npm install && npm run dev

# Or both at once:
./dev.sh
```

Providers are configured either in the root `.env` (see `.env.example`) or per-session in the Settings tab. With no key set, the deterministic engine still runs fully.

11.2 Tests & evaluation

```
cd backend
python -m pytest -q                # 73 passed / 1 skipped (offline, fast)
python -m eval.detection --no-ai    # deterministic regression gate (reproducible, no
key)                                key)
python -m eval.detection            # AI-on live run (spends tokens)
RUN_AI_EVAL=1 python -m pytest eval/ # opt-in agent-recovery test (needs a provider)
```

11.3 Containers

A `docker-compose.yml` brings up the backend and the built frontend (served by nginx). Dockerfiles + `.dockerignore` are in the repo.

11.4 Key operational tunables (env)

Variable	Effect
<code>DATABASE_URL</code>	Swap SQLite → Postgres
<code>CAPABILITY_LLM_ENABLED=0</code>	Force regex-only classification (no LLM spend)
<code>DETECTION_AGENT_ENABLED=0</code>	Fall back to single-shot detection
<code>LLM_MIN_INTERVAL_SEC</code> / per-provider / per-model	Throttle spacing (set 0 on a paid tier)
<code>LLM_REQUEST_TIMEOUT_SEC</code> · <code>LLM_MAX_RETRIES</code>	Per-request timeout & retry budget
<code>RATE_LIMIT_ANALYZE_PER_HOUR</code> · <code>_CHAT_PER_MINUTE</code>	Inbound spend guards
<code>DETECTION_AGENT_MAX_ITERS</code> · <code>_TOKEN_BUDGET</code>	Detection loop cost caps
<code>PATH_W_*</code> · <code>DISC_SCORE_*</code>	Re-tune path/discovery ranking weights

12 · Where we can improve

The current stack is the right stack for a single-operator lab tool. The ceiling is not the framework choice — it's **process-local state and the absence of identity**. These are the honest gaps, grouped by priority.

12.1 Blocking gaps for any shared / production use

Gap	Why it matters	Recommended fix
HIGH No authentication	Anyone who reaches the API can trigger LLM spend and read every enterprise's data. The IP rate limiter is a stopgap, not a control.	OIDC login with a Backend-For-Frontend (httpOnly session), roles (viewer/analyst/admin), append-only audit log.
HIGH Process-local state	<code>api/state.py</code> globals and the rate-limiter dict live in one process. Can't scale horizontally or survive a restart mid-run. This is the real production ceiling.	Move truth to Postgres , and queue/locks/rate-limits/pub-sub to Redis .
MED Pipeline runs in-request	A long analysis run is tied to a thread/queue bridge in the web process; a deploy or crash loses it.	Move runs to arq workers; progress → a <code>run_events</code> table + Redis pub/sub; SSE becomes replay-then-tail with <code>Last-Event-ID</code> .
MED Provider keys in the app	Every provider key sits in the app's env; no per-team budgets or usage isolation.	Put a LiteLLM Proxy in front as the sole LLM egress: virtual keys, per-team budgets; provider keys leave the app entirely.

A full 5-phase migration plan already exists (foundations → Postgres/Redis externalization → identity/RBAC/audit → worker extraction + LLM proxy → k8s/observability/hardening). It keeps FastAPI / React / litellm — the stack is fine; the *statefulness* is what's evicted.

12.2 Test & quality gaps

- **Frontend has ~0 automated tests.** The backend has 73; the SPA relies on manual/headless checks. A component/integration test layer (Vitest + Testing Library, plus a Playwright smoke of the core flows) should gate any production push.
- **No Alembic migrations.** The DB is created via `create_all`; schema evolution is currently additive-by-necessity (data folded into JSON blobs to avoid migrations). Adopt Alembic before the schema grows.
- **Edge-level detection recall (~0.5)** has room: the agent finds the right targets but not always the exact documented route. Better hop-substitution scoring, or teaching the agent to enumerate multiple routes per target, would lift it.

12.3 Data-realism gaps

- **EPSS/KEV are hand-enriched static tables** keyed by CVE. For real use, pull EPSS live from FIRST.org and KEV live from CISA (documented as a caveat in the methodology). This is the single biggest step toward using the platform on a real scan.
- **No live scanner/asset ingestion.** Everything is static CSV. A connector to a real scanner (Qualys/Tenable/Nessus export) and a real CMDB/CI source would make it operational rather than

demonstrative.

- **CCF is uniformly 1.0** across the lab data (no verified compensating controls modeled). Real deployments should feed verified WAF/MFA/EDR/segmentation evidence to earn the reductions.

12.4 LLM-layer opportunities

- **Determinism vs. free-tier reality.** Even at temperature 0, Mistral isn't perfectly reproducible, which wobbles the AI-on metrics. Caching detection-loop results (as capability classification already does) would recover reproducibility and cut cost.
- **Model-to-task fit.** Roles are assigned to spread free-tier load, not purely to match model strength to task difficulty. On a paid tier, deliberately routing the hardest reasoning (detection, correlation) to the strongest model — and easy structured extraction to a cheap fast one — is a straightforward win.
- **Prompt-injection surface.** Finding descriptions and CMDB fields flow into prompts. If real scan data (attacker-influenced) is ever ingested, the grounding fence protects *facts* but the narration text should be treated as untrusted and sandboxed accordingly.

What is already right — don't rebuild it

The deterministic-first architecture, the graph-grounding fence (0 hallucination), the provider abstraction, the eval harness, and the held-out oracle are the hard, valuable parts and they are sound. The improvements above are about **operationalizing** a proven design, not correcting it.

13 · Appendix

13.1 Glossary

Term	Meaning
GRS	Ghrab Risk Score — the context-aware 0–100 risk score replacing naive CVSS sorting.
CMDB	Configuration Management Database — the relational model of assets (CIs), zones, reachability, ownership.
CI	Configuration Item — one asset (server, database, network device, cloud resource).
ACW	Asset Criticality Weight (0–1) — how much an asset matters to business + regulators.
TCM	Toxic Combination Score (0–10) — how many crown jewels a finding opens a path to, and how directly.
EPSS	Exploit Prediction Scoring System — probability a CVE is exploited in the wild.
KEV	CISA Known Exploited Vulnerabilities catalog — binary "confirmed exploited" gate.
Crown jewel	A business-critical target asset (e.g. SWIFT gateway, settlement system, regulated DB).
Attack path / chain	A multi-hop route INTERNET → crown jewel; the thing the platform discovers.
Oracle	The held-out documented ground-truth paths — used only to grade detection, never fed to it.
ReAct loop	Reason → Act → Observe agent loop: the detection agent reasons, calls a graph tool, observes the real result, iterates.
Grounded / plausible / rejected	The three verification labels a detected path can carry (see §6.2).
DORA / CIF	EU Digital Operational Resilience Act; Critical or Important Function — triggers the 30-day SLA cap.
DiGraph	Directed graph (networkx) — the reachability model paths are enumerated on.

13.2 Agent → provider/model reference (current deployment)

Role	Provider	Model	Temp	Rationale
capability	Mistral	mistral-small-2506	0	High-volume batched structured extraction
correlation	Mistral	mistral-medium-2505	0	Cross-finding reasoning (toxic combos)
attack_path	Mistral	mistral-medium-2505	0	Core chain reasoning + detection loop
remediation	Mistral	mistral-small-2506	0	Interactive, low-latency per-finding
compliance	Mistral	mistral-small-2506	0	Structured framework mapping
triage	Mistral	mistral-medium-2505	0.2	Prose synthesis + chat (documented non-zero)

13.3 Source-of-truth documents in the repo

File	Contents
docs/AGENTS.md	Agent-by-agent API breakdown; the Phases 0–5 subsystem, tool contract, scoring formula, eval how-to.
CHANGES.md	The full Phases 0–5 change log with committed metrics.
docs/attack-path-intelligence-brief.md	The implementation brief that specified the detection upgrade.
docs/cmdb-accuracy-brief.md	The CMDb-accuracy initiative (relational conversion + validation).
data/ghrab_risk_methodology.md	The complete GRS methodology with worked examples.
README.md	Two-service quick start.